

Software Maintenance & Configuration Management

Not from your notes — drafted fresh. Your SE Fundamentals topic mentioned that software "deteriorates" through repeated changes, and named Software Configuration Management as one of the umbrella activities — this topic fills in the detail behind both of those mentions, since maintenance classification and SCM process both come up frequently in interviews.

Part 1: Software Maintenance

1. What Is Software Maintenance?

Software maintenance is the process of modifying software *after* it has been delivered, to correct faults, improve performance or other attributes, or adapt it to a changed environment. This directly connects back to the SE Fundamentals topic's point that software doesn't wear out physically, but *deteriorates* — every one of these modifications is a chance to introduce a new defect, which is exactly why "each change can increase the failure rate" over a system's lifetime.

2. The Four Types of Maintenance

Type	Trigger	Example
Corrective	A defect/bug found <i>after</i> release	Fixing a crash that occurs when a user enters a negative quantity
Adaptive	The <i>environment</i> around the software changes	Updating code because the OS, a third-party API, or a legal/regulatory requirement changed
Perfective	Users request improvements or new capabilities	Adding a new report format, or improving a slow query's performance
Preventive	Proactively reducing <i>future</i> maintenance cost/risk, before a specific problem forces it	Refactoring a fragile module, updating stale documentation, upgrading a dependency before it's end-of-life

Corrective → fixing something that's WRONG
Adaptive → keeping up with something that CHANGED (outside the software)
Perfective → making something BETTER (that already works)
Preventive → stopping something from becoming a problem LATER

A commonly-tested fact: industry studies consistently find **Perfective maintenance is the largest single category** — often cited as roughly 50–65% of total maintenance effort — because once software is in production and providing value, users and stakeholders keep finding new capabilities they want, and this demand doesn't taper off the way corrective-maintenance demand does (which shrinks over time as more bugs get fixed). This is worth remembering specifically because it surprises people who assume "maintenance" mostly means "bug fixing" — corrective maintenance is usually a *minority* of the total effort, not the majority.

3. Why Maintenance Is Expensive

- **Ripple effects from poor design** — low cohesion and high coupling (see the Design Concepts topic) mean a small change in one place breaks things elsewhere, turning a simple fix into a large, risky one.
- **Loss of original context** — the developers who originally wrote the code, and understood *why* certain decisions were made, may have moved on; without good documentation, every change requires re-deriving intent from the code alone.
- **Legacy constraints** — old code is often built on outdated assumptions, frameworks, or platforms that make even small changes disproportionately difficult.
- **Fear of regressions** — without a solid regression test suite (see the Software Testing topic), every change carries the risk of silently breaking something that used to work, which slows maintenance down as teams compensate with extra manual caution.

Maintainability as a measurable attribute: metrics like Cyclomatic Complexity (see the Metrics & Cost Estimation topic) are used as *proxies* for how expensive a module will be to maintain — a module with high cyclomatic complexity isn't just harder to test initially, it's also harder for a future maintainer to safely reason about when making a change months or years later.

Part 2: Software Configuration Management (SCM)

4. What Is SCM?

Software Configuration Management is the umbrella activity (named directly in the SE Fundamentals topic's list of umbrella activities) responsible for managing the effects of change

throughout the software process. Unlike a framework activity that's concentrated at one point in the timeline, SCM runs continuously — because requirements, design, and code can all change at any point in a project's life, not just during one designated "change phase."

5. Software Configuration Items (SCIs)

An **SCI** is any work product placed under formal configuration control — not just source code. This includes: requirements documents, design documents, source code, test plans, user manuals, and even the build/deployment scripts. Anything that other work depends on, and that needs to be tracked and versioned, is a candidate SCI.

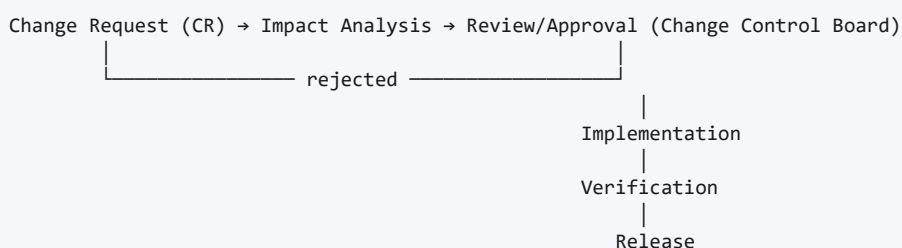
6. Baselines

A **baseline** is a formally reviewed and approved version of an SCI (or a set of SCIs) that serves as the basis for further development — and, critically, can only be changed afterward through a **formal change control procedure**, not by anyone editing it freely.

Draft → Reviewed → Approved (BASELINE) → any further change requires
a formal Change Request

Baselines exist specifically to prevent the chaos of an artifact changing invisibly out from under everyone relying on it — once a requirements document is baselined, for instance, a developer building against it can trust it won't silently shift underneath them; any legitimate need to change it goes through the visible, tracked change-control process below.

7. The Change Control Process



1. **Change Request (CR)** — someone formally proposes a change to a baselined item.
2. **Impact Analysis** — before approving, the team assesses what else the change would affect (which other SCIs, which components, how much effort, what risk).
3. **Review/Approval** — often via a **Change Control Board (CCB)**, a designated group (which may include technical leads, project managers, and sometimes customer representatives) who weigh the impact analysis and either approve, reject, or defer the change.
4. **Implementation** — the approved change is actually made.

5. **Verification** — confirming the change was implemented correctly and didn't introduce unintended side effects (this is where regression testing, from the Software Testing topic, does its job).
6. **Release** — the change is folded into a new baseline/version, and the cycle can repeat.

Why does a CR need Impact Analysis *before* approval, rather than just implementing changes as requested? Because a change that looks small in isolation (e.g., "just rename this field") can have ripple effects across every other SCI or component that referenced it — exactly the ripple-effect problem that low coupling is meant to minimize at the code level, and that Impact Analysis is meant to catch at the *process* level before the change is even made, rather than discovering the ripple effects only after something downstream breaks.

8. Version Control (the mechanism behind SCM)

Version control systems are the practical tooling that makes SCM enforceable in day-to-day development:

- **Repository** — the stored history of all versions of the tracked files.
- **Commit** — a recorded snapshot of a set of changes, with a message explaining what and (ideally) why.
- **Branch** — an independent line of development, letting work happen in isolation before being merged back.
- **Merge** — combining changes from one branch into another.

Centralized (e.g., SVN) vs. Distributed (e.g., Git): in a centralized system, there's one authoritative central repository that everyone commits directly to; in a distributed system, every developer has a full copy of the entire history locally, and synchronizes with others (and with a shared "central" remote, by convention) via push/pull — distributed systems dominate modern practice because they allow offline work, faster local operations, and much cheaper branching.

9. Configuration Audits

Once changes are made, two kinds of audits verify the software configuration is actually correct and complete:

- **Functional Configuration Audit (FCA)** — verifies that the *functionality* of the delivered SCI actually matches what was specified (does the software do what the requirements say it should?).
- **Physical Configuration Audit (PCA)** — verifies that *all* the required deliverables and documentation are actually present and match what's supposed to have been produced (is everything that should have been delivered actually there — code, docs, test results, etc.?).

FCA vs. PCA, one line: FCA checks "does it work as specified" (functional correctness); PCA checks "is everything that was supposed to be delivered actually here" (completeness of the deliverable set).

Interview Questions With Answers

Q1. Classify each of the following as Corrective, Adaptive, Perfective, or Preventive maintenance, and justify each: (a) fixing a null-pointer crash reported by a user, (b) updating code because a payment provider deprecated their old API version, (c) adding a dark-mode UI option because users requested it, (d) refactoring a tangled module before adding a planned new feature to it.

Answer: (a) is **Corrective** maintenance, because it fixes a defect — something the software was already supposed to do correctly but didn't — discovered after release. (b) is **Adaptive** maintenance, because nothing was wrong with the code itself; the *external environment* (the payment provider's API) changed, forcing the software to adapt to keep working at all. (c) is **Perfective** maintenance, because dark mode isn't fixing a defect or responding to an external environment change — it's an enhancement, a new capability requested to make the software better than it already was. (d) is **Preventive** maintenance, because the refactor isn't fixing a currently-reported bug or responding to an already-planned feature's specific requirements — it's proactively reducing future risk/cost (making the module safer to extend) *before* that risk materializes into an actual problem during the upcoming feature work.

Q2. Why is Perfective maintenance typically the largest category of maintenance effort, and why might this surprise someone who assumes "maintenance" mainly means bug-fixing?

Answer: Perfective maintenance dominates because the demand for it doesn't diminish over a system's lifetime the way corrective-maintenance demand does — as a system matures and more of its defects get found and fixed, the corrective-maintenance workload naturally shrinks, but user and business demand for new capabilities, improvements, and enhancements tends to continue indefinitely as long as the software remains in active use and stakeholders keep discovering new value it could provide. This surprises people who intuitively associate "maintenance" with "keeping something from breaking," since that framing implicitly assumes maintenance is fundamentally reactive (respond to problems) — when in reality, most maintenance effort on a healthy, actively-used system is proactive/value-adding rather than defect-driven, closer in spirit to ongoing feature development than to firefighting.

Q3. Explain what a "baseline" is in SCM and why an SCI can't simply be edited freely once it's baselined.

Answer: A baseline is a formally reviewed and approved version of a software configuration item that becomes the trusted reference point other work builds on top of — for example, once a requirements document is baselined, developers can start designing and coding against it, trusting that its content won't shift unpredictably underneath them. If a baselined SCI could simply be edited freely afterward, anyone relying on it would have no reliable way to know whether the version they're working from is still accurate, since it could have silently changed without their knowledge — this would reintroduce exactly the kind of chaos SCM exists to prevent. Requiring a formal change control procedure instead means any change to a baseline is visible, deliberately reviewed for its downstream impact, and communicated to everyone depending on that baseline, rather than happening invisibly.

Q4. Why does the change control process include a separate "Impact Analysis" step before a Change Control Board approves a request, rather than the CCB just directly deciding to approve or reject?

Answer: A Change Control Board making a decision about whether to approve a change needs to actually understand what that change would affect beyond the immediate item being modified — which other SCIs, components, or already-planned work depend on the thing being changed, how much effort the change realistically requires, and what risk it introduces. Without a dedicated Impact Analysis step producing that information first, the CCB would effectively be approving or rejecting changes based on an incomplete picture, likely relying only on how the request *sounds* rather than what it actually *entails* — which risks either rejecting genuinely low-risk changes out of unwarranted caution, or approving changes whose real downstream cost and risk weren't understood until after the fact, defeating the entire purpose of having a formal review step in the first place.

Q5. What's the practical difference between a Functional Configuration Audit and a Physical Configuration Audit, and can a delivered SCI pass one while failing the other?

Answer: A Functional Configuration Audit checks whether the software's actual behavior matches what was specified — essentially, does it do what it's supposed to do — while a Physical Configuration Audit checks whether everything that was supposed to be delivered (code, documentation, test results, user manuals, and so on) is actually present and complete, independent of whether the software itself works correctly. Yes, an SCI set can absolutely pass one while failing the other: software could function perfectly correctly according to its specification (passing FCA) while missing required documentation or an incomplete test report (failing PCA); conversely, every required document and deliverable could technically be present and accounted for (passing PCA) while the software itself doesn't actually behave as the specification requires (failing FCA) — which is exactly why both audits are performed as genuinely separate checks rather than treating one as implying the other.

Q6. Why is SCM described as an "umbrella activity" rather than a phase that happens once, early or late in a project?

Answer: SCM is described as an umbrella activity because changes to requirements, design, and code can legitimately need to happen at any point across the entire project timeline, not just during one designated phase — a requirements document might need a formally-controlled change during what's nominally the "construction" phase, just as code might need a controlled change discovered during "deployment." If SCM were treated as a one-time phase, there would be no formal mechanism to manage changes that arise outside that phase's boundaries, which given how frequently change actually occurs throughout real projects, would leave most of the project's actual change activity completely uncontrolled. Running SCM continuously, alongside every framework activity, is what actually lets baselines, change requests, and audits function as a coherent system across a project's full lifetime rather than only protecting whatever happened to exist at one fixed point in time.

Q7. Why do modern teams overwhelmingly favor distributed version control (like Git) over centralized version control (like older SVN setups), from an SCM perspective?

Answer: In a centralized system, every meaningful version-control operation (committing, viewing history, branching) requires communicating with the single central server, which means developers can't work effectively offline and every operation carries network latency; a distributed system gives every developer a complete local copy of the full history, making most operations (commits, history browsing, even branch creation) instantaneous and possible without any network connection at all. From an SCM perspective specifically, distributed systems also make branching dramatically cheaper, which encourages teams to isolate in-progress, not-yet-baselined work on its own branch far more freely — supporting the baseline concept more naturally, since work-in-progress never has to touch the shared, already-baselined mainline until it's actually ready to be reviewed and merged in, rather than centralized systems' historical tendency to make branching heavyweight enough that teams often worked directly against a shared trunk instead.

Q8. Scenario: A production incident is traced to a config file change that was made directly on the production server, without going through any change request, review, or version control — nobody else on the team knew the change had been made until the system broke. Which SCM concepts were violated, and what should have happened instead?

Answer: This violates several SCM concepts at once: the config file should have been treated as a Software Configuration Item under version control in the first place, meaning any change to it would automatically be tracked and visible rather than silently made outside any recorded system; the production configuration should have been a baseline, meaning a change to it required a formal Change Request rather than being editable directly by anyone with server access; and the change bypassed the entire change control process (no impact analysis assessing what the config change might affect, no review/approval from a Change Control Board or equivalent, no verification step confirming the change was correct before it took effect). What should have happened instead is that the config file lives in

version control alongside the rest of the codebase, any proposed change goes through a Change Request with impact analysis (which might have specifically caught the issue that caused the incident before it reached production), gets reviewed and approved, and only then gets deployed through a controlled release process — at which point the change is also visible to the whole team, rather than being a surprise only discovered once the system had already broken.